

Programming

Interpret teacher-provided solution requirements to design and develop a software solution using an object-oriented programming language.

The big picture

This outcome is the full build of a small object-oriented program. You read the requirements, constraints and scope; design with mock-ups, IPO charts and pseudocode; then develop using the language's features — data types, data structures, control structures, and the OOP principles of abstraction and encapsulation. Good practice runs through all of it: meaningful naming conventions, internal documentation, validation, and disciplined debugging and testing. The dot points map onto the problem-solving methodology: analysis, design, development and evaluation.

How it's assessed

You are assessed on precise programming vocabulary and on reasoning about code, not just writing it. Expect to define data types and structures, trace or complete pseudocode using sequence/selection/iteration, explain abstraction vs encapsulation, justify naming conventions, write or complete a test table with expected vs actual results, and choose validation techniques. Vague answers ("it makes the code better") score poorly; tie every point to what the code actually does.

Key knowledge at a glance

KK	Topic	In one line
KK01	Emerging trends & AI in programming	Programming used to mean typing every line yourself. Now you can describe what you want in plain English and a model writes the code. That changes how we build software — but not whether we still have to understand it. This module shows you the trends, the tools, and the one skill examiners care about most: judging whe
KK02	Characteristics of data types	When you declare a variable, you're not just naming a box — you're signing a contract that fixes three things at once.
KK03	Data structures	One-dimensional arrays, lists and records — the three ways a program groups related data together, and how to tell them apart in an exam.
KK04	Requirements, Constraints & Scope	Before a single line of code is written, four questions decide whether your software succeeds: what must it do, how well must it do it, what limits hem you in, and where is the edge of the job. Read them wrong and you build the wrong app perfectly.

KK	Topic	In one line
KK06	Who owns what you make — and what you use ?	Every game, app and website is built from created things : code, artwork, music, fonts, fragments of other people's libraries. The moment you make something original, the law treats it as yours. The moment you reach for someone else's work, the law asks one question — do you have permission? This module is the ruleboo
KK07	The two ideas that tame complexity	A real app has thousands of moving parts. Nobody can hold all of it in their head at once. OOP's trick is to break the system into objects — self-contained units that each look after one thing — so you only ever think about a small piece at a time.
KK08	The four building blocks every program is made from	A computer can only ever do a handful of truly different things: store a value, decide between paths, repeat work, calculate with operators, and reuse a named block of code. Master these and you can read or write any object-oriented program ever written.
KK09	Naming conventions for solution elements	A name is the smallest design decision in your program — and the one you make most often. Get it right and your code reads like plain English. Get it wrong and your own project, six weeks later, reads like a stranger's handwriting. This dot point names two conventions you must know: Hungarian notation and camelCase.
KK10	Internal documentation The notes a programmer leaves inside the code — for the next person, who is often future-you.	Kai, a Year 12 student, built CourtBook : a small app that lets students book the school gym's basketball courts at lunch. It worked. Then Kai graduated and left.
KK11	Validation techniques	A reasonableness check on input — not a truth check.
KK12	Debugging & Testing	Writing code that runs is the easy half. Proving it does the right thing — and tracking down why it doesn't — is the skill examiners reward. This module covers test tables, constructing test data, and the debugging techniques every VCE programmer is expected to name and use.

Chapter glossary

Term	Meaning
Abstraction	Hiding complexity by exposing only the essential features of an object.
Encapsulation	Bundling data and methods together and restricting direct access to the data.
Data structure	An organised store of related values, e.g. a one-dimensional array, list or record.
Control structure	The flow of a program: sequence, selection (if) and iteration (loops).

Term	Meaning
Camel case	Naming convention: first word lowercase, each later word capitalised (e.g. totalScore).
Hungarian notation	Naming convention using a type prefix (e.g. strName, intCount).
Internal documentation	Comments and naming inside the code that explain its structure and logic.
Validation	Existence, type and range checking of input before it is used.
Test table	A table of test cases listing input, expected output and actual output.

Answering across the chapter

These scenario questions pull together several key-knowledge points at once — the kind of integrated thinking the exam rewards. Compare the weak and strong responses.

Q1. Apply · 4 marks

A program reads a class's test scores into an array and must report the highest score and reject any score outside 0–100. Outline the data structure, one validation technique and one control structure you would use, and justify each.

✗ A weak answer

Use a list and an if statement and a loop to do it. (→ Names structures but justifies nothing and ties to no detail — 1.)

✓ A strong answer

A **one-dimensional array** stores the scores because they are many values of the same type accessed by index. **Range checking** rejects any score below 0 or above 100 before it is stored, protecting data quality. **Iteration** (a loop) steps through the array comparing each score to find the maximum. Each choice fits the task: array for a list of like values, range check for the stated 0–100 rule, loop to process every element.

How the marks are awarded

- 1 mark — array chosen and justified.
- 1 mark — range checking chosen and justified.
- 1 mark — iteration chosen and justified.
- 1 mark — answer is tied to the 0–100 / find-highest task.

Q2. Explain · 3 marks

Explain how good naming conventions and internal documentation make a solution easier to maintain.

✗ A weak answer

They make the code tidy so it's easier to read later. (→ *One idea repeated; no link to maintenance by another person — 1.*)

✓ A strong answer

Meaningful names following a convention (e.g. *customerTotal* in camel case) make each variable self-documenting, so a maintainer understands its purpose without tracing the whole program. Internal documentation — comments explaining why a section works as it does — records intent that the code alone cannot show. Together they let a different programmer modify the solution confidently and with fewer errors.

How the marks are awarded

- 1 mark — self-documenting names aid understanding.
- 1 mark — comments record intent/logic.
- 1 mark — links to maintenance by another programmer.

Chapter revision checklist

- State functional/non-functional requirements, constraints and scope for a solution.
- Select appropriate data types and a data structure (array, list, record).
- Represent a design with a mock-up, IPO chart and pseudocode/flowchart.
- Explain and apply abstraction and encapsulation.
- Use sequence, selection and iteration correctly in pseudocode.
- Apply naming conventions (camel case, Hungarian notation) and internal documentation.
- Choose existence, type and range checking for inputs.
- Build a test table and use breakpoints/output to debug.